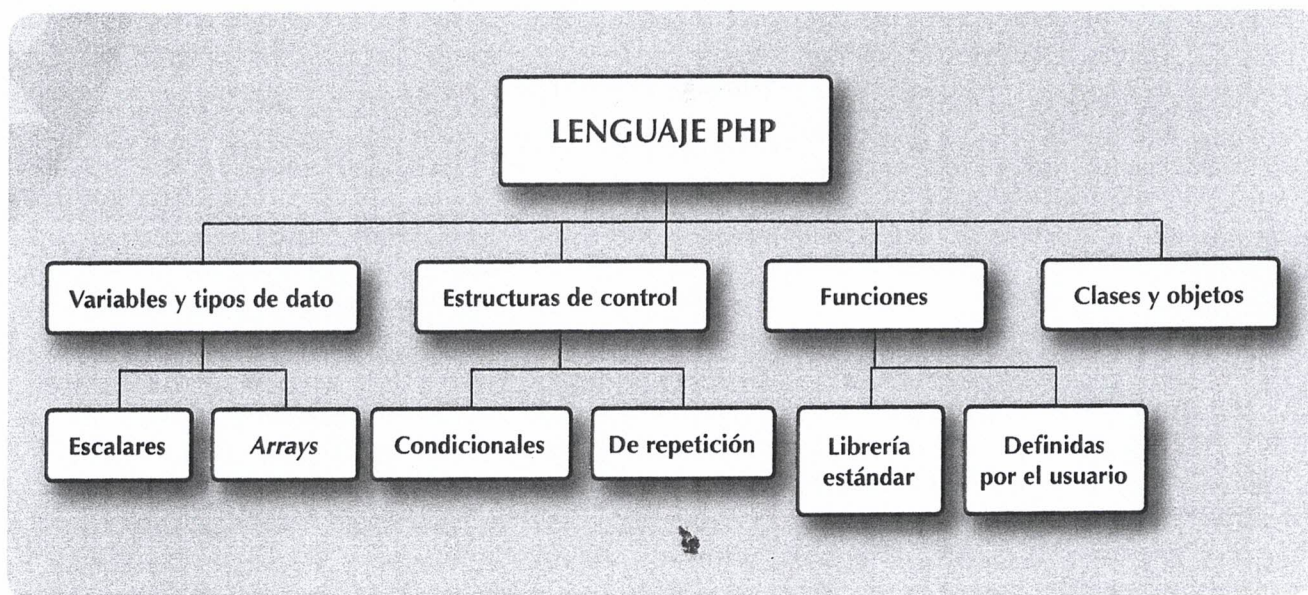


Introducción al lenguaje PHP

Objetivos

- ✓ Conocer la sintaxis básica de PHP.
- ✓ Entender cómo se integran PHP y HTML.
- ✓ Describir los tipos de datos existentes en PHP.
- ✓ Manejar las estructuras de control básicas.
- ✓ Aprender a utilizar los *arrays* asociativos.
- ✓ Presentar la notación de objetos en PHP.

Mapa conceptual



Glosario

Array. Es un tipo de dato compuesto habitual en los lenguajes de programación. Aunque los detalles varían entre lenguajes, en general, se parecen a vectores o listas ordenadas.

Bucle. Estructura de programación que permite repetir instrucciones.

Estructura condicional. Posibilita ejecutar o no una instrucción según se cumpla una condición.

Función. Conjunto de instrucciones que realiza una tarea concreta.

Librería. Las funciones relacionadas entre sí se agrupan en librerías o bibliotecas.

Programación orientada a objetos. Paradigma de programación basado en la idea de objetos, elementos que agrupan variables y funciones.

Script. Programa sencillo, habitualmente ejecutado por un intérprete en lugar de ser compilado.

Variable. Posición en la memoria del ordenador identificada por un nombre. La variable almacena datos. Estos datos son el valor de la variable.

2.1. PHP y HTML. Código incrustado

PHP es el lenguaje de programación para desarrollo web en el lado del servidor. Desde su aparición en 1994 ha tenido gran aceptación y se puede decir que es lenguaje más extendido para el desarrollo en el lado del servidor. Aunque no es la única opción, lo normal es que el intérprete de PHP sea un módulo del servidor web.

El lenguaje PHP es flexible y permite programar pequeños *scripts* con rapidez. Comparado con lenguajes como Java, requiere escribir menos código y, en general, resulta menos engorroso. La sintaxis de los elementos básicos es bastante parecida a la de lenguajes muy extendidos como Java y C. Por estos motivos, es un lenguaje rápido de aprender para las personas con alguna experiencia en programación.

En este capítulo se presentan la sintaxis y los elementos básicos del lenguaje PHP. Se espera que el lector esté familiarizado con los conceptos básicos de programación estructurada y orientada a objetos.

En el desarrollo web es muy habitual utilizar PHP incrustado dentro ficheros HTML. El código PHP se introduce dentro del HTML utilizando la etiqueta `<?php` para abrir el bloque de PHP y la etiqueta `?>` para cerrarlo.

El ejemplo **hola_mundo.php** muestra una página HTML completa con un bloque PHP incrustado en las líneas 7-10. El bloque tiene una única sentencia que sirve para mostrar la cadena "Hola mundo".

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <title>Hola mundo</title>
5      </head>
6      <body>
7          <?php
8              echo "Hola mundo";
9          ?>
10     </body>
11 </html>
```

Al solicitar la página al servidor web, el resultado es:

Hola mundo

Si se consulta el código fuente de la página (pulsando Ctrl + u), se obtiene:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Hola mundo</title>
    </head>
    <body>
        Hola mundo
    </body>
</html>
```

Se puede observar que el servidor web ha modificado una parte del fichero. El bloque PHP ha desaparecido y en su lugar aparece "Hola mundo". El servidor web procesa los bloques de PHP y los sustituye por su salida, es decir, por lo que muestran las sentencias del bloque. En este caso se utiliza `echo`, que muestra el valor de una variable o de una cadena de texto.

**TOMA NOTA**

Para probar los ejemplos del libro hay que instalar el entorno de trabajo como se explica en el capítulo 1. Para probar este ejemplo accede a http://localhost/cap2/hola_mundo.php. Para el resto de los ejemplos solo hay que cambiar el nombre del fichero. Recuerda arrancar el servidor web.

2.2. Sintaxis de PHP

El elemento básico en PHP es el bloque. Un bloque PHP está delimitado por las etiquetas correspondientes y contiene una serie de sentencias separadas por punto y coma.

```
<?php
    sentencial;
    sentencian;
? >
```

RECUERDA

- ✓ Cuando un fichero contiene solo PHP se recomienda no cerrar la etiqueta del último bloque. Puede dar problemas si la respuesta del servidor involucra varios ficheros.

2.3. Variables y tipos de dato

Una de las características principales de PHP es que es un lenguaje *no fuertemente tipado*. Esto quiere decir que no es necesario indicar el tipo de dato al declarar una variable. De hecho, las variables no se declaran, se crean la primera vez que se les asigna un valor. El tipo de dato depende del valor con que se inicialicen.

Esto agiliza la escritura de programas, pero también tiene inconvenientes. Si no se presta atención, puede dar lugar a código de baja calidad y, a medida que las aplicaciones crecen, pueden darse errores difíciles de depurar.

2.3.1. Declaración de variables

En PHP los identificadores de las variables van siempre precedidos por el carácter '\$'. El identificador de la variable debe comenzar por una letra o un guion bajo ('_'), y puede estar formado por números, letras y guiones bajos.

Para declarar una variable, solo hay que asignarle un valor:

```
$nombre = valor;
```

Por ejemplo, esta sentencia declara la variable `$entero`, que será de tipo `integer` porque se inicializa con un entero.

```
$entero = 4;
```

También es posible cambiar el tipo de dato de una variable simplemente asignándole un valor de otro tipo de dato, como se puede ver en el ejemplo **tipos_dato.php** (líneas 8-12). El ejemplo utiliza la función `gettype()`, que devuelve el tipo de dato de una variable.

```
1  <?php
2  /* declaración de variables */
3  $entero = 4; // tipo integer
4  $numero = 4.5; // tipo coma flotante
5  $cadena = "cadena"; // tipo cadena de caracteres
6  $bool = TRUE; //tipo booleano
7  /* cambio de tipo de una variable */
8  $a = 5; // entero
9  echo gettype($a); // imprime el tipo de dato de a
10 echo "<br>";
11 $a = "Hola"; // cambia a cadena
12 echo gettype($a); // se comprueba que ha cambiado
```

La salida del ejemplo confirma que la variable cambia de tipo de dato.

```
integer
string
```

2.3.2. Asignación por copia y por referencia

En principio, la asignación de variables se realiza mediante copia. Es decir, si hacemos:

```
$a = $b;
```

se crea una nueva variable *a* y se le asigna el valor que tenga *b*. Las variables *a* y *b* representan posiciones diferentes de memoria, aunque tengan el mismo valor después de la asignación.

También es posible definir una referencia a una variable utilizando el operador *ampersand*:

```
$var2 = &$var1;
```

En este caso `$var2` no es una nueva variable con el valor de `$var1`. Por el contrario, `$var2` apunta a la misma dirección de memoria que `$var1`, de manera que `$var1` y `$var2` son en realidad dos nombres para el mismo dato. En el ejemplo **copia.php** se muestra que, al cambiar el valor de una referencia, se modifica también el de la variable referenciada.

```
<?php
    $var1 = 100;
    $var2 = &$var1; // asignación por referencia
```

```

$var3 = $var1; // asignación por copia
echo "$var2<br>"; // muestra 100
$var2 = 300; // cambia el valor de $var2
echo "$var1<br>"; // $var1 también cambia
$var3 = 400; // este cambio no afecta a $var1
echo $var1;

```

La salida de este ejemplo será:

```

100
300
300

```

2.3.3. Variables no inicializadas

Si se intenta utilizar una variable antes de asignarle un valor, se genera un error de tipo E_NOTICE, pero no se interrumpe la ejecución del *script*. Si una variable no inicializada aparece dentro de una expresión, dicha expresión se calcula tomando el valor por defecto para ese tipo de dato. En el ejemplo **no_init.php**, se puede ver lo que ocurre al utilizar una variable no inicializada dentro de una expresión.

```

1 <?php
2 $var1 = 100;
3 $var3 = 100 + $var2; // $var2 no existe, se toma como 0
4 echo "$var3 <br>"; // muestra 100
5 $var3 = 100 * $var2; // $var2 no existe, se toma como 0
6 echo "$var3 <br>"; // muestra 0

```

En la línea 3, se intenta sumar una variable no inicializada. Como el valor por defecto para un entero es 0, el resultado de la suma es 100. En la línea 5, se intenta multiplicar por una variable no inicializada y, al multiplicar por 0, el resultado es 0.

La salida muestra un mensaje de error por cada intento de utilización de una variable no inicializada.

```

Notice: Undefined variable: var2 in C:\xampp\htdocs\cap2\ejecinicializaciones.php on line 3
100
Notice: Undefined variable: var2 in C:\xampp\htdocs\cap2\ejecinicializaciones.php on line 5
0

```

2.3.4. Constantes

Para definir constantes se utiliza la función `define()`, que recibe el nombre de la constante y el valor que queremos darle.

```
define("LIMITE", 1000);
```

Es habitual utilizar identificadores en mayúsculas para las constantes.

2.3.5. Tipos de datos escalares

PHP ofrece cuatro tipos de datos escalares: *integer*, *float*, *boolean* y *string*.

A) Números

Para representar números enteros se usa el tipo de dato *integer*. El tamaño y los valores máximo y mínimo de un entero dependen de la plataforma, y se pueden conocer mediante los constantes `PHP_INT_SIZE`, `PHP_INT_MAX` y `PHP_INT_MIN`, respectivamente.

Para números reales, se utiliza el tipo *float*. El tamaño también depende de la plataforma, pero suele ofrecer una precisión de 14 decimales. En cualquier caso, la precisión de los *float* presenta los mismos problemas que en otros lenguajes y los redondeos pueden dar sorpresas.

La conversión entre *integer* y *float* es automática. Si se recibe un *float* cuando se espera un *integer*, se trunca. Si al realizar una operación sobre un entero el resultado supera los valores límite o tiene decimales, se convierte a *float*. También se pueden utilizar los operadores de conversión, `(int)` o `(float)`.

El ejemplo **tipos_numericos.php** muestra las diferentes opciones disponibles para literales *integer* o *float* y algunas operaciones entre ellos.

```
<?php
echo PHP_INT_SIZE.'<br>';
echo PHP_INT_MIN.'<br>';
echo PHP_INT_MAX.'<br>';
$a = 0b100; // en binario
$a = 0100; // octal
$a = 0x100; // hexadecimal
$a = 3/2; // la división entre enteros no da problemas
echo $a.'<br>'; // 1.5
$b = 7.5;
$a = (int)$b; //casting a int
echo $a.'<br>'; // 7, se trunca
$b = 7e2; // notación científica
$b = 7E2;
```

B) Cadenas

El tipo de dato *string* permite almacenar cadenas de caracteres. Para delimitar una cadena es posible utilizar comillas simples o dobles, pero hay una diferencia. Si se utilizan comillas dobles (también llamadas *comillas mágicas*), las variables que aparezcan dentro de la cadena se sustituirán por su valor. Las comillas dobles son muy prácticas, ya que es más rápido insertar directamente las variables que montar las cadenas con varias concatenaciones.

```
<?php
$var = "Paco";
$a = "Hola $var <br>";
$b = 'Hola $var';
echo $a;
echo $b;
```

La salida será:

Hola Paco

Hola \$var

Hola Paco

En el primer caso, \$var se sustituye por su valor, "Paco". En el segundo, se interpreta como texto normal. La última línea del *script* muestra el operador de concatenación entre cadenas, ".".



TOMA NOTA

A la hora de formatear la salida hay que tener en cuenta que esta va a ser procesada como HTML por un navegador web, es decir, los saltos de línea se ignoran y los espacios en blanco consecutivos colapsan. Los caracteres de escape como '\n', '\r' y '\t' son ignorados por el navegador. Para introducir un salto de línea la opción más sencilla es incluir la etiqueta de salto de línea de HTML ("
") en la salida, como se puede ver en los ejemplos anteriores.

C) Booleanos

Este tipo de dato es para variables *booleanas*. Solo pueden tomar los valores TRUE y FALSE, verdadero y falso. Este es el tipo de dato que se obtiene, entre otros casos, como resultado de los operadores de comparación y se utiliza en sentencias condicionales y bucles.

Cuando se espera un valor *booleano* y se recibe otro tipo de dato, se aplican las reglas de conversión recogidas en el cuadro 2.1.

CUADRO 2.1

Conversión implícita a *boolean*

Tipo	Valor como <i>boolean</i>
integer	Si es 0 se toma FALSE, en otro caso como TRUE
float	Si es 0.0 se toma FALSE, en otro caso como TRUE
string	Si es una cadena vacía o "0", se toma como FALSE, en otro caso como TRUE
variables no inicializadas	FALSE
null	FALSE
array	Si no tiene elementos se toma FALSE, en otro caso como TRUE

D) Otros tipos de dato

Además de los tipos de datos escalares en PHP también existen los siguientes tipos de datos:

1. *array*. Para representar colecciones de elementos. Los *arrays* de PHP son muy potentes y se explican con detalle en el apartado 2.2.5.
2. *object*, PHP tiene soporte completo para la programación orientada a objetos. Se explica en el apartado 2.2.8.
3. *callable*. Un tipo de dato especial para representar funciones de *callback*, funciones que se pasan a otras funciones.
4. *null*. El tipo de dato *null* representa una variable que no ha sido asignada. Solo puede tomar un único valor, NULL. Se considera que una variable es de tipo *null* si se le asigna el valor NULL o no está inicializada.
5. *resource*. Este tipo de dato representa recursos externos, como una conexión a una base de datos.

CUADRO 2.2
Tipos de dato en PHP

Tipo	Descripción
integer	Números enteros
float	Números reales en coma flotante
string	Cadenas de caracteres
boolean	Booleanos, TRUE o FALSE
array	Colección de elementos identificados
object	Un objeto es una instancia de una clase
callable	Para las funciones de <i>callback</i>
null	Para representar variables no asignadas
resource	Representa recursos externos

2.3.6. Ámbito de las variables

El ámbito de una variable es la parte del código en que esta es visible. Una variable declarada en un fichero PHP está disponible en ese fichero y en los ficheros que se incluyan desde este.

Por otro lado, las funciones definen un ámbito local, de manera que las variables que se declaran en las mismas solo son accesibles desde la propia función. Además, desde la función no se puede acceder a otras variables que no sean las locales o sus argumentos.

Para definir variables globales hay dos opciones. La palabra reservada `global` y la variable predefinida `$_GLOBALS`. Las variables globales son accesibles desde cualquier función o fichero de la aplicación.

2.3.7. Variables predefinidas

En PHP hay muchas variables predefinidas disponibles. Contienen información sobre el servidor, datos enviados por el cliente o variables de entorno. Dentro de las variables predefinidas

hay un grupo de ellas, las *superglobales*, que están disponibles en cualquier ámbito. Cada una de ellas guarda información de un tipo.

Por ejemplo, en `$_SERVER` hay información sobre el servidor en el que está alojada la página. El script **global_server.php** muestra algunos de los datos disponibles.

```
<?php
echo "Ruta dentro de htdocs: " . $_SERVER['PHP_SELF'];
echo "Nombre del servidor: " . $_SERVER['SERVER_NAME'];
echo "Software del servidor: " . $_SERVER['SERVER_SOFTWARE'];
echo "Protocolo: " . $_SERVER['SERVER_PROTOCOL'];
echo "Método de la petición: " . $_SERVER['REQUEST_METHOD'];
```

Las variables *superglobales* (cuadro 2.3) son muy relevantes para el desarrollo de aplicaciones web y se irán poniendo en práctica a lo largo del libro.

CUADRO 2.3
Variables superglobales

Nombre	Descripción
<code>\$GLOBALS</code>	Variables globales definidas en la aplicación
<code>\$_SERVER</code>	Información sobre el servidor
<code>\$_GET</code>	Parámetros enviados con el método GET (en la URL)
<code>\$_POST</code>	Parámetros enviados con el método POST (formularios)
<code>\$_FILES</code>	Ficheros subidos al servidor
<code>\$_COOKIE</code>	Cookies enviadas por el cliente
<code>\$_SESSION</code>	Información de sesión
<code>\$_REQUEST</code>	Contiene la información de <code>\$_GET</code> , <code>\$_POST</code> y <code>\$_COOKIE</code>
<code>\$_ENV</code>	Variables de entorno

2.4. Comentarios

En PHP se pueden utilizar comentarios:

- De bloque, encerrados entre `/*` y `*/`.
- De línea, comenzando por `//` o por `#`.

```
<?php
//comentario de línea
$a = 0; // comentario de línea
/* comentario
de bloque
*/
# comentario de una sola línea
```

Como en cualquier lenguaje, comentar adecuadamente el código se considera una buena práctica de programación.

2.5. Estructuras de control

PHP cuenta con las estructuras de control habituales en la programación estructurada para realizar sentencias condicionales y de repetición. La sintaxis es muy parecida a la de Java o C.

2.5.1. Estructuras condicionales

Las estructuras condicionales de PHP son `if`, `if-else`, `if-elseif` y `switch`.

La sentencia condicional más sencilla es la estructura `if`. La sintaxis general es:

```
if (condición)
    instrucción
```

Si se cumple la condición, se ejecuta la instrucción. Si no se cumple, no se ejecuta. Para agrupar dentro del `if` más de una sentencia, se encierran entre llaves.

```
if (condición){
    instrucción 1
    instrucción n
}
```

La condición es una expresión que se evalúa a verdadero o falso, siguiendo las normas de conversión a *boolean* si es necesario.

En el siguiente ejemplo solo se cumple el segundo `if` y, por tanto, la salida del programa sería "Es mayor que cero".

```
<?php
$var = 3;
if($var < 0) echo "Es menor que cero";
if ($var > 0){
    echo "Es mayor que cero";
}
```

Cuando se utiliza un `if` se puede añadir un `else`. Las instrucciones dentro del `else` solo se ejecutan cuando la condición del `if` no se cumple.

```
<?php
$var = 3;
if($var < 0){
    echo "Es menor que cero";
}else{
    echo "Es mayor o igual que cero";
}
```

Si se anidan varias sentencias condicionales, se puede usar `elseif`, que es equivalente a `else if`.

```
<?php
    $var = 3;
    if ($var == 1) {
        echo "Es un uno";
    }elseif ($var == 2) {
        echo "Es un dos";
    }elseif ($var == 3) {
        echo "Es un tres";
    }else{
        echo "No es un uno, ni un dos, ni un tres"
    }
}
```

La primera condición que se cumpla es la que se ejecuta. Si no se cumple ninguna, se ejecuta el `else` final (si lo hay).

Para agrupar varios `if` puede ser útil la estructura `switch`, también habitual en otros lenguajes. El ejemplo `switch.php` es equivalente al anterior, pero más fácil de leer.

```
<?php
    $var = 3;
    switch($var){
        case 1:
            echo "Es un 1";
            break;
        case 2:
            echo "Es un 2";
            break;
        case 3:
            echo "Es un 3";
            break;
        default:
            echo "No es un 1, ni un 2, ni un 3";
    }
}
```

Según el valor de `$var`, se ejecutará un `case` u otro. La sección `default` se ejecuta cuando no se da ninguno de los `case`.

RECUERDA

- ✓ Si no hay un `break` al final de un `case`, la ejecución continúa con el siguiente.

2.5.2. Estructuras de repetición

Las estructuras de repetición o bucles sirven para repetir un conjunto de instrucción mientras se dé una condición. PHP cuenta con las estructuras habituales: `for`, `while` y `do-while`, que tienen la misma sintaxis que en Java o C.

Para el bucle `for`, la sintaxis es:

```
for(instrucciones de inicialización; condición; instrucciones de iteración) {
    instrucciones del bucle;
}
```

Las instrucciones de inicialización se realizan una única vez al llegar al bucle. Las instrucciones dentro del cuerpo del bucle se repetirán mientras se cumpla la condición.

Después de ejecutar las instrucciones de inicialización se evalúa la condición. Si se cumple, se ejecutan las instrucciones del bucle y las instrucciones de iteración. Después se vuelve a comprobar la condición. El proceso se repite hasta que la condición deja de cumplirse y a partir de ahí la ejecución continúa con las instrucciones que estén después del bucle.

El ejemplo **bucle_for.php** muestra un bucle `for` básico que se repite cinco veces. La variable `$i` se inicializa a cero y su valor se incrementa en uno tras cada iteración. El bucle se repite mientras `$i` valga menos que cinco, es decir, de cero a cuatro.

```
<?php
    for($i = 0; i < 5; $i = $i + 1){
        echo "$i <br>";
    }
```

El bucle `while` también sigue la sintaxis habitual:

```
while (condición) {
    instrucciones;
}
```

Igual que con el `for`, las instrucciones se ejecutan mientras se cumpla la condición. El `while` no cuenta con instrucciones de inicialización o iteración, pero se pueden añadir antes y al final del bucle, respectivamente. El ejemplo **bucle_while.php** tiene la misma salida que el anterior.

```
<?php
    $i = 0;
    while($i < 5){
        echo "$i <br>";
        $i = $i +1;
    }
```

El bucle `do-while` es similar, pero la condición se evalúa después de ejecutar las ejecuciones del bucle.

```
do {
    instrucciones;
} while (condición);
```

Este ejemplo es equivalente a los anteriores.

```
<?php
    $i = 0;
    do{
        echo "$i <br>";
        $i = $i +1;
    } while ($i < 5);
```

El bucle `do-while` se diferencia de los anteriores en que las instrucciones dentro del bucle se ejecutan por lo menos una vez, como se puede ver en el siguiente ejemplo:

```
<?php
    $i = 0;
    do {
        echo "En el do-while: $i <br>";
        $i = $i + 1;
    } while ($i < 0);
    while ($i < 0) {
        echo "En el while: $i <br>";
        $i = $i + 1;
    }
```

La salida de este ejemplo será:

En el do-while: 0

Dentro de un bucle es posible usar las sentencias `break` y `continue`. La primera sirve para salir del bucle o `switch` en el que aparece. En el ejemplo **break.php**, el bucle acaba antes de llegar a cinco porque al llegar a tres se ejecuta el `break`.

```
<?php
    $i = 0;
    while ($i < 5){
        echo "$i <br>";
        $i++; // es lo mismo que $i = $ i + 1;
        if ($i == 3){
            break;
        }
    }
```

En PHP la sentencia `break` admite un número, que representa el número de niveles de anidación de los que debe salir. Así se puede salir de un bucle anidado utilizando una única secuencia `break` como se puede ver a continuación.

```
<?php
    echo "Primer for anidado: <br>";
    for ($i = 0; $i < 3; $i++) {
        for ($j = 0; $j < 3; $j++) {
            echo "i: $i j: $j <br>";
            if ($j == 1) {
                break; //es lo mismo que poner break 1
            }
        }
    }
    echo "Segundo for anidado: <br>";
    for ($i = 0; $i < 3; $i++) {
        for ($j = 0; $j < 3; $j++) {
            echo "i: $i j: $j <br>";
            if ($j == 1){
                break 2;
            }
        }
    }
}
```

En el primer bucle anidado se utiliza un `break` sin pasarle ningún número, que es lo mismo que pasarle un uno, y por tanto solo sale del bucle interior. El bucle exterior se repetirá tres veces y el interior seis, dos por cada vez que se ejecute el exterior, con los valores de $j=0$ y $j=1$. En el segundo bucle anidado, al ejecutarse la línea `x` se acaban los dos bucles. Esto ocurre en la primera iteración del bucle externo y la segunda del interno. La salida será:

Primer for anidado:

```
i: 0 j: 0
i: 0 j: 1
i: 1 j: 0
i: 1 j: 1
i: 2 j: 0
i: 2 j: 1
```

Segundo for anidado:

```
i: 0 j: 0
i: 0 j: 1
```

Esto también se aplica a la sentencia condicional `switch`. Por ejemplo, si un `switch` está dentro de un bucle, utilizando `break 2` se sale de ambos.

```
<?php
    $i = 0;
    echo "Primer switch anidado: <br>";
    while ($i < 2) {
        switch ($i) {
            case 0:
                echo "Es un cero <br>";
                break;
            case 1:
                echo "Es un uno <br>";
                break;
        }
        $i++;
    }
    $i = 0;
    echo "Segundo switch anidado: <br>";
    while ($i < 2) {
        switch ($i) {
            case 0:
                echo "Es un cero <br>";
                break 2;
            case 1:
                echo "Es un uno <br>";
                break;
        }
        $i++;
    }
}
```

La salida de este ejemplo es:

Primer switch anidado:

Es un cero

Es un uno

Segundo switch anidado:

Es un cero

La sentencia **continue** fuerza una nueva iteración del bucle. Las instrucciones que estén dentro del bucle, pero después de **continue** no se ejecutan y, si se cumple la condición del bucle, se ejecuta una nueva iteración. Si se trata de un bucle **for**, se ejecutan las instrucciones de autoincremento antes de evaluar la condición.

En el ejemplo **continue.php** se puede observar el funcionamiento de **continue**.

```
<?php
for($i = 0; $i < 5; $i = $i + 1){
    if ($i == 3){
        continue;
    }
    echo "$i <br>";
}
```

Cuando **\$i** vale tres se ejecuta el **continue**, por lo que el **echo** a continuación no ejecuta esa iteración. La salida será:

0
1
2
4

Actividad propuesta 2.1



Escribe un programa que calcule el factorial de un número. Recuerda que el factorial solo está definido para números enteros mayores o iguales que cero.

2.5.3. Otras estructuras de control

Es posible incluir otros ficheros utilizando las sentencias **include** y **require**.

```
include "mifichero.php";
require "mifichero.php";
```

Se diferencian solo en el tratamiento de errores. Si no se encuentra el fichero especificado, **include** genera un error de tipo **E_NOTICE**, pero **require** genera un error **E_FATAL**, lo que implica el fin de la ejecución del *script*. Si no hay error, en ambos casos el fichero especificado se

ejecuta. Si el fichero es una librería con funciones, estas se declaran y quedan disponibles para el fichero que lo incluye. Si el fichero contiene sentencias fuera de una función, estas se ejecutan.

El ejemplo **requerir.php** incluye otro fichero.

```
<?php
$a = "variable del principal";
require "ejerequerido.php";
$b = "otra variable del principal";
echo "En el script principal";
```

El fichero requerido es el siguiente:

```
<?php
echo "En el fichero requerido <br>";
echo $a;
echo $b;
```

Al solicitar al servidor **cap2/requerir.php** se obtiene esta salida:

En el fichero requerido

variable del principal

Notice: Undefined variable: b in C:\xampp\htdocs\cap2\ejerequerido.php on line 4

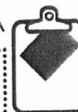
En el script principal

Se puede observar que:

- Las instrucciones del fichero requerido se ejecutan.
- Las variables del fichero principal están disponibles en el requerido si se habían inicializado antes de requerirlo. Esto ocurre con `$a`, pero no con `$b`, que se inicializa después y, por tanto, se muestra un mensaje de error. Es decir, el ámbito de las variables del fichero principal incluye los ficheros requeridos.

También están disponibles las sentencias `require_once` e `include_once`, análogas a las anteriores, pero con la diferencia de que solo se ejecutan los ficheros especificados si no han sido incluidos o requeridos con anterioridad.

TOMA NOTA



La inclusión de ficheros tiene una diferencia con otros lenguajes. Supongamos que el fichero A incluye al fichero B. Las rutas relativas que aparezcan en B se interpretarán a partir del directorio de A.

Para solucionarlo, en el fichero B se utiliza `dirname(__FILE__)` que devuelve la ruta del fichero:

```
include(dirname(__FILE__)."\".'fichero.php');
```

Para acabar con las estructuras de control, la sentencia `return` finaliza la ejecución del fichero cuando se encuentra fuera de una función. En el siguiente ejemplo, el `echo` final no se llega a ejecutar, porque se ha ejecutado antes el `return`.

```
<?php
    $a = 0;
    if ($a == 0){
        return;
    }
    echo "Después del if";
```

Si el *script* se estaba ejecutando mediante un `require` o `include`, la ejecución continúa en el fichero que lo incluyó. Si se pasa como argumento un número entero, este se devuelve como valor de retorno.

Si se encuentra dentro de una función, finaliza la ejecución de esta y la función devuelve el argumento del `return`. La ejecución continua en el fichero o función que llama a la función.

2.6. Operadores

PHP cuenta con los operadores habituales para operaciones aritméticas, lógicas, de manipulación de cadenas y demás.

Entre los operadores de comparación cabe señalar los operadores `===` y `!==`, llamados Idéntico y No Idéntico, que no están presentes en todos los lenguajes. El operador Idéntico se usa para comparar dos expresiones y se evalúa como verdadero cuando las dos expresiones tienen el mismo valor y además el mismo tipo de dato. Se diferencia del operador Igual, `==`, en que este, cuando las expresiones no tienen el mismo tipo de dato, intenta convertirlas antes de compararlas. El operador Idéntico es útil para evitar sorpresas inesperadas en la conversión de datos. En el ejemplo **identico.php** se puede comprobar la diferencia.

```
<?php
    $a = 3;
    $b = "3";
    if ($a == $b){
        echo "Son iguales <br>";
    }else{
        echo "No son iguales <br>";
    }
    if ($a === $b){
        echo "Son idénticos <br>";
    }else{
        echo "No son idénticos <br>";
    }
```

La primera comparación permite la conversión de tipos, y la cadena "3" se convierte al entero 3. La salida será:

Son iguales

No son idénticos